
DBMS – Project

“A Unified Platform for Modern Court Management”

❖ Group Members

No.	Name	Student ID
1.	Sherathiya Darpan	202401199
2.	Deep Shobhashana	202401205
3.	Prins Satapara	202401188
4.	Tirth Solanki	202401211
5.	Shaurya Pratap	202401197

Queries and their solutions for each different scenarios:

1. Case Centered Queries

→ These queries are mainly centered with case entity. All these queries are complex and lengthy but meaningful in perspective of our database.

1) Find criminal cases that have already reached the framing of charges stage in their hearings but where the Case_document table has no chargesheet document recorded. Include the judge, the prosecutor, and how many days have passed since the charges were framed.

```
SELECT
    c.Case_ID,
    c.CNR,
    c.Case_title,
    c.Case_type,
    h.Hearing_ID,
    cl.Date                                AS charges_framed_date,
    cl.Courtroom_number,

    j.Judge_ID,
    u_j.Name                             AS judge_name,
    j.Working_status                     AS judge_status,

    h.Prosecutor_ID                     AS prosecutor_bar_no,
    u_p.Name                             AS prosecutor_name,
    lp.Firm                             AS prosecutor_firm,

    h.Verdict                           AS charges_framed_text,

    (CURRENT_DATE - cl.Date)            AS days_since_charges_framed
```

```

FROM Hearing      h
JOIN "Case"      c  ON  c.Case_ID          = h.Case_ID
JOIN Cause_list  cl ON  cl.List_ID         = h.List_ID
JOIN Judge       j  ON  j.Judge_ID         = h.Judge_ID
JOIN "User"      u_j ON u_j.User_ID        = j.User_ID
JOIN Lawyer      lp ON lp.Bar_council_number = h.Prosecutor_ID
JOIN "User"      u_p ON u_p.User_ID        = lp.User_ID

WHERE
    h.Purpose      = 'Framing of Charges'
    AND c.Case_type = 'Criminal'

    AND NOT EXISTS (
        SELECT 1
        FROM    Case_document cd
        WHERE   cd.Case_ID      = h.Case_ID
                AND cd.Document_type = 'Charge Sheet'
    )

ORDER BY days_since_charges_framed DESC;

```

2) Identify cases where the presiding judge's area of specialisation has zero overlap with the case type, how many hearings have happened, and the judge's actual specialisations.

```

SELECT
    c.Case_ID,
    c.CNR,
    c.Case_title,
    c.Case_type,
    c.Current_status,

    u.Name                               AS Judge_name,
    j.Working_status                     AS Judge_working_status,

    STRING_AGG(DISTINCT js.Specialisation, ', ' ORDER BY js.Specialisation)
    AS Judge_specialisations,

    COUNT(DISTINCT h.Hearing_ID)        AS Total_hearings

FROM "Case" c

```

```

JOIN Hearing h
    ON h.Case_ID = c.Case_ID

JOIN Judge j
    ON j.Judge_ID = h.Judge_ID

JOIN "User" u
    ON u.User_ID = j.User_ID

JOIN Judge_specialisation js
    ON js.Judge_ID = j.Judge_ID

WHERE NOT EXISTS (
    SELECT 1
    FROM Judge_specialisation js2
    WHERE js2.Judge_ID = j.Judge_ID
    AND js2.Specialisation LIKE '%' || c.Case_type || '%')

GROUP BY
    c.Case_ID,
    c.CNR,
    c.Case_title,
    c.Case_type,
    c.Current_status,
    j.Judge_ID,
    u.Name,
    j.Working_status

ORDER BY Total_hearings DESC;

```

3) Find all active cases — both civil and criminal — where a litigant appears as a party in the case but has no lawyer assigned on their side. Include how many hearings have already taken place without them being represented.

```

SELECT
    c.Case_ID,
    c.CNR,
    c.Case_title,
    c.Case_type,
    c.Current_status,

```

```

        li.Litigant_ID,
        u.Name                                AS Litigant_name,
        li.Type                              AS Litigant_type,
        COUNT(DISTINCT h.Hearing_ID)        AS Hearings_without_representation

FROM "Case" c

JOIN Case_litigants cli
    ON cli.Case_ID = c.Case_ID

JOIN Litigant li
    ON li.Litigant_ID = cli.Litigant_ID

JOIN "User" u
    ON u.User_ID = li.User_ID

LEFT JOIN Hearing h
    ON h.Case_ID = c.Case_ID
    AND h.Verdict IS NOT NULL
    OR (
        h.Case_ID = c.Case_ID
        AND h.Next_date IS NOT NULL
    )

WHERE c.Current_status = 'Active'

    AND c.Case_type IN ('Civil', 'Criminal')

    AND NOT EXISTS (
        SELECT 1
        FROM Case_lawyers cla
        WHERE cla.Case_ID = c.Case_ID
        AND cla.Type      = li.Type
    )

GROUP BY
    c.Case_ID,
    c.CNR,
    c.Case_title,
    c.Case_type,
    c.Current_status,
    li.Litigant_ID,
    u.Name,

```

```

li.Type

HAVING COUNT(DISTINCT h.Hearing_ID) >= 0

ORDER BY Hearings_without_representation DESC, c.Case_type;

```

4) Identify cases that were filed more than a year ago, have had zero hearings scheduled so far, have no evidence on record, and no interlocutory applications. Include the filing lawyer from the e-filing record, and the litigant who initiated it.

```

SELECT
    c.Case_ID,
    c.CNR,
    c.Case_title,
    c.Case_type,
    c.Current_status,
    c.Filing_date,
    (CURRENT_DATE - c.Filing_date) AS Days_since_filing,

    ul.Name AS Filing_lawyer,
    lw.Firm AS Lawyer_firm,

    up.Name AS Initiating_litigant,
    li.Type AS Litigant_type

FROM "Case" c

JOIN E_filing ef
    ON ef.Petitioner_ID IN (
        SELECT cli.Litigant_ID
        FROM Case_litigants cli
        WHERE cli.Case_ID = c.Case_ID
    )

JOIN Lawyer lw
    ON lw.Bar_council_number = ef.Petitioner_lawyer_ID

JOIN "User" ul
    ON ul.User_ID = lw.User_ID

JOIN Case_litigants cli
    ON cli.Case_ID = c.Case_ID

```

```
JOIN Litigant li
    ON li.Litigant_ID = cli.Litigant_ID

JOIN "User" up
    ON up.User_ID = li.User_ID

WHERE c.Filing_date < CURRENT_DATE - INTERVAL '1 year'

    AND NOT EXISTS (
        SELECT 1
        FROM Hearing h
        WHERE h.Case_ID = c.Case_ID
    )

    AND NOT EXISTS (
        SELECT 1
        FROM Evidence e
        WHERE e.Case_ID = c.Case_ID
    )

    AND NOT EXISTS (
        SELECT 1
        FROM Interlocutory_applications ia
        WHERE ia.Case_ID = c.Case_ID
    )

GROUP BY
    c.Case_ID,
    c.CNR,
    c.Case_title,
    c.Case_type,
    c.Current_status,
    c.Filing_date,
    ul.Name,
    lw.Firm,
    up.Name,
    li.Type

ORDER BY Days_since_filing DESC;
```

5) Find witnesses who have appeared in hearings for more than one case, Show which lawyer called them in each case and what the hearing outcome was.

```
SELECT
    w.Witness_ID,
    w.Name                                AS Witness_name,
    w.Witness_type,
    w.Gender,

    c.Case_ID,
    c.CNR,
    c.Case_title,
    c.Case_type,

    h.Hearing_ID,
    h.Purpose                              AS Hearing_purpose,
    h.Verdict                            AS Hearing_outcome,

    up.Name                              AS Prosecutor_name,
    ud.Name                              AS Defense_lawyer_name,

    multi.Total_cases_appeared

FROM Witness w

JOIN Has_witness hw
    ON hw.Witness_ID = w.Witness_ID

JOIN Hearing h
    ON h.Hearing_ID = hw.Hearing_ID

JOIN "Case" c
    ON c.Case_ID = h.Case_ID

LEFT JOIN Lawyer lp
    ON lp.Bar_council_number = h.Prosecutor_ID

LEFT JOIN "User" up
    ON up.User_ID = lp.User_ID
```

```

LEFT JOIN Lawyer ld
    ON ld.Bar_council_number = h.Defense_lawyer_ID

LEFT JOIN "User" ud
    ON ud.User_ID = ld.User_ID

JOIN (
    SELECT hw2.Witness_ID,
           COUNT(DISTINCT hh.Case_ID) AS Total_cases_appeared
    FROM Has_witness hw2
    JOIN Hearing hh
    ON hh.Hearing_ID = hw2.Hearing_ID
    GROUP BY hw2.Witness_ID
    HAVING COUNT(DISTINCT hh.Case_ID) > 1
) AS multi
    ON multi.Witness_ID = w.Witness_ID

ORDER BY w.Witness_ID, c.Case_ID;

```

6) For cases that appear in Case_history at more than one court level, find those where the verdict at the lower court was a conviction or dismissal, but the verdict at the higher court was either an acquittal or a settlement in favour of the other party.

```

SELECT
    c.Case_ID,
    c.CNR,
    c.Case_title,
    c.Case_type,
    c.Current_status,

    lower.Court_ID           AS Lower_court,
    lower.Court_type         AS Lower_court_type,
    lower.Verdict            AS Lower_court_verdict,
    lower.Date               AS Lower_court_date,

    higher.Court_ID          AS Higher_court,
    higher.Court_type         AS Higher_court_type,
    higher.Verdict           AS Higher_court_verdict,
    higher.Date              AS Higher_court_date

FROM "Case" c

```

```

JOIN Case_history lower
  ON lower.Case_ID = c.Case_ID
  AND lower.Verdict IN ('Conviction', 'Dismissal')

JOIN Case_history higher
  ON higher.Case_ID = c.Case_ID
  AND higher.Verdict IN ('Acquittal', 'Settlement')
  AND higher.Date > lower.Date

WHERE (
  SELECT COUNT(DISTINCT ch.Court_ID)
  FROM Case_history ch
  WHERE ch.Case_ID = c.Case_ID
) > 1

ORDER BY c.Case_ID, lower.Date;

```

7) Find judges whose average gap between their consecutive hearings across all those cases exceeds 90 days.

```

SELECT
  j.Judge_ID,
  u.Name AS Judge_name,
  j.Working_status,

  COUNT(DISTINCT h.Case_ID) AS Total_cases,
  COUNT(h.Hearing_ID) AS Total_hearings,
  ROUND(AVG(gap.Days_gap), 2) AS Avg_gap_between_hearings_days

FROM Judge j

JOIN "User" u
  ON u.User_ID = j.User_ID

JOIN Hearing h
  ON h.Judge_ID = j.Judge_ID

JOIN (

```

```

SELECT
h1.Judge_ID,
h1.Hearing_ID,

(MIN(h2.Next_date) - c11.Date)    AS Days_gap

FROM Hearing h1
JOIN Cause_list c11
ON c11.List_ID = h1.List_ID
JOIN Hearing h2
ON h2.Judge_ID = h1.Judge_ID
AND h2.Case_ID = h1.Case_ID
JOIN Cause_list c12
ON c12.List_ID = h2.List_ID
AND c12.Date > c11.Date
GROUP BY h1.Judge_ID, h1.Hearing_ID, c11.Date
) AS gap
ON gap.Judge_ID = j.Judge_ID
AND gap.Hearing_ID = h.Hearing_ID

GROUP BY
j.Judge_ID,
u.Name,
j.Working_status

HAVING AVG(gap.Days_gap) > 90

ORDER BY Avg_gap_between_hearings_days DESC;

```

8) Find all dates where two or more hearings were scheduled in the same courtroom at overlapping or identical time slots.

```

SELECT
    c11.Date                AS Hearing_date,
    c11.Courtroom_number,
    c11.Time                AS Slot_1_time,
    c12.Time                AS Slot_2_time,

    c1.CNR                 AS Case_1_CNR,
    c1.Case_title           AS Case_1_title,
    h1.Hearing_ID           AS Hearing_1_ID,

```

```

        h1.Purpose                                AS Hearing_1_purpose,

        c2.CNR                                  AS Case_2_CNR,
        c2.Case_title                           AS Case_2_title,
        h2.Hearing_ID                           AS Hearing_2_ID,
        h2.Purpose                                AS Hearing_2_purpose

FROM Cause_list cl1

JOIN Cause_list cl2
    ON cl2.Courtroom_number = cl1.Courtroom_number
   AND cl2.Date              = cl1.Date
   AND cl2.Time              = cl1.Time
   AND cl2.List_ID           > cl1.List_ID

JOIN Hearing h1
    ON h1.List_ID = cl1.List_ID

JOIN Hearing h2
    ON h2.List_ID = cl2.List_ID

JOIN "Case" c1
    ON c1.Case_ID = h1.Case_ID

JOIN "Case" c2
    ON c2.Case_ID = h2.Case_ID

ORDER BY
    cl1.Date,
    cl1.Courtroom_number,
    cl1.Time;

```

9) Among all cases currently under trial, find those whose last hearing ended with a non-final verdict, where the next scheduled hearing date is more than 6 months away from that last hearing.

```

SELECT
    c.Case_ID,
    c.CNR,
    c.Case_title,
    c.Case_type,

```

```

        c.Current_status,

        h.Hearing_ID           AS Last_hearing_ID,
        h.Purpose                AS Last_hearing_purpose,
        h.Verdict              AS Non_final_verdict,

        cl.Date                AS Last_hearing_date,
        h.Next_date            AS Next_scheduled_date,

        (h.Next_date - cl.Date) AS Days_until_next_hearing,

        u.Name                 AS Judge_name

FROM "Case" c

JOIN Hearing h
    ON h.Case_ID = c.Case_ID

JOIN Cause_list cl
    ON cl.List_ID = h.List_ID

JOIN Judge j
    ON j.Judge_ID = h.Judge_ID

JOIN "User" u
    ON u.User_ID = j.User_ID

WHERE c.Current_status = 'Under Trial'

    AND h.Verdict NOT IN ('Conviction', 'Acquittal', 'Dismissed',
'Settlement')
    AND h.Verdict IS NOT NULL

    AND (h.Next_date - cl.Date) > 180

    AND cl.Date = (
        SELECT MAX(cl2.Date)
        FROM Hearing h2
        JOIN Cause_list cl2
            ON cl2.List_ID = h2.List_ID
        WHERE h2.Case_ID = c.Case_ID
    )

```

```
ORDER BY Days_until_next_hearing DESC;
```

10) For every case filed in the last two years, calculate the number of days between the case filing date and the date of the very first hearing. Find cases where this gap exceeded 180 days

```
SELECT
    c.Case_ID,
    c.CNR,
    c.Case_title,
    c.Case_type,
    c.Current_status,
    c.Filing_date,

    first_h.First_hearing_date,

    (first_h.First_hearing_date - c.Filing_date) AS
Days_to_first_hearing,

    u.Name AS Judge_name,
    ul.Name AS Filing_lawyer

FROM "Case" c

JOIN (
    SELECT
        h.Case_ID,
        MIN(cl.Date) AS First_hearing_date
    FROM Hearing h
    JOIN Cause_list cl
    ON cl.List_ID = h.List_ID
    GROUP BY h.Case_ID
) AS first_h
    ON first_h.Case_ID = c.Case_ID

JOIN Hearing h
    ON h.Case_ID = c.Case_ID

JOIN Cause_list cl
    ON cl.List_ID = h.List_ID
```

```

        AND cl.Date          = first_h.First_hearing_date

JOIN Judge j
    ON j.Judge_ID          = h.Judge_ID

JOIN "User" u
    ON u.User_ID           = j.User_ID

JOIN E_filing ef
    ON ef.Petitioner_ID IN (
        SELECT cli.Litigant_ID
        FROM Case_litigants cli
        WHERE cli.Case_ID = c.Case_ID
    )

JOIN Lawyer lw
    ON lw.Bar_council_number = ef.Petitioner_lawyer_ID

JOIN "User" ul
    ON ul.User_ID           = lw.User_ID

WHERE c.Filing_date >= CURRENT_DATE - INTERVAL '2 years'

    AND (first_h.First_hearing_date - c.Filing_date) > 180

GROUP BY
    c.Case_ID,
    c.CNR,
    c.Case_title,
    c.Case_type,
    c.Current_status,
    c.Filing_date,
    first_h.First_hearing_date,
    u.Name,
    ul.Name

ORDER BY Days_to_first_hearing DESC;

```

11) Find all cases where no hearing has been scheduled yet.

```
SELECT
    c.Case_ID,
    c.CNR,
    c.Case_title,
    c.Case_type,
    c.Current_status,
    c.Filing_date,
    (CURRENT_DATE - c.Filing_date)      AS Days_since_filing

FROM "Case" c

WHERE NOT EXISTS (
    SELECT 1
    FROM Hearing h
    WHERE h.Case_ID = c.Case_ID
)

ORDER BY Days_since_filing DESC;
```

12) For each case type find the average number of hearings held per case and the average days from filing to first hearing.

```
SELECT
    c.Case_type,

    COUNT(DISTINCT c.Case_ID)      AS Total_cases,

    ROUND(
        COUNT(h.Hearing_ID) * 1.0 /
        NULLIF(COUNT(DISTINCT c.Case_ID), 0)
        , 2)                        AS Avg_hearings_per_case,

    ROUND(
        AVG(
            (first_h.First_hearing_date - c.Filing_date)
        )
        , 2)                        AS Avg_days_to_first_hearing

FROM "Case" c
```

```

LEFT JOIN Hearing h
    ON h.Case_ID = c.Case_ID

LEFT JOIN (
    SELECT
        h2.Case_ID,
        MIN(c1.Date)                AS First_hearing_date
    FROM Hearing h2
    JOIN Cause_list c1
    ON c1.List_ID = h2.List_ID
    GROUP BY h2.Case_ID
) AS first_h
    ON first_h.Case_ID = c.Case_ID

GROUP BY
    c.Case_type

ORDER BY Avg_hearings_per_case DESC;

```

2. workload scenario:

→ These queries are based on lawyers , judges and Admin's workload

1) Show how many hearings each judge conducted per month with adjournment rate.

```

SELECT j.Judge_ID, u.Name AS judge_name,
TO_CHAR(c1.Date, 'YYYY-MM') AS month,
COUNT(h.Hearing_ID) AS total_hearings,
SUM(CASE WHEN LOWER(h.Verdict) LIKE '%adjourn%' THEN 1 ELSE 0 END)
AS adjourned,
ROUND(
100.0 * SUM(CASE WHEN LOWER(h.Verdict) LIKE '%adjourn%'
THEN 1 ELSE 0 END)
/ NULLIF(COUNT(h.Hearing_ID), 0), 2
) AS adjournment_rate_pct
FROM Hearing h
JOIN Cause_list c1 ON c1.List_ID = h.List_ID
JOIN Judge j ON j.Judge_ID = h.Judge_ID

```

```
JOIN "User" u ON u.User_ID = j.User_ID
GROUP BY j.Judge_ID, u.Name, TO_CHAR(c1.Date, 'YYYY-MM')
ORDER BY j.Judge_ID, month;
```

2) Find the top 5 most active lawyers by number of cases they are involved in.

```
SELECT l.Bar_council_number, u.Name AS lawyer_name,
COUNT(DISTINCT cl.Case_ID) AS total_cases
FROM Lawyer l
JOIN "User" u ON u.User_ID = l.User_ID
JOIN Case_lawyers cl ON cl.Bar_council_number = l.Bar_council_number
GROUP BY l.Bar_council_number, u.Name
ORDER BY total_cases DESC
LIMIT 5;
```

3) Find judges who have active cases assigned but have conducted zero hearings.

```
SELECT j.Judge_ID, u.Name AS judge_name,
COUNT(DISTINCT cj.Case_ID) AS assigned_active_cases
FROM Judge j
JOIN "User" u ON u.User_ID = j.User_ID
JOIN Case_judges cj ON cj.Judge_ID = j.Judge_ID
JOIN "Case" c ON c.Case_ID = cj.Case_ID
WHERE c.Current_status = 'Active'
AND j.Judge_ID NOT IN (
SELECT DISTINCT Judge_ID FROM Hearing
)
GROUP BY j.Judge_ID, u.Name
ORDER BY assigned_active_cases DESC;
```

4) Find admins who have a rejection rate above 30 percent on e-filings and list how many filings they have rejected.

```
SELECT adm.Admin_ID, u.Name AS admin_name,
COUNT(ef.Diary_number) AS total_filings,
SUM(CASE WHEN ef.Status = 'Rejected' THEN 1 ELSE 0 END) AS rejected,
ROUND(
100.0 * SUM(CASE WHEN ef.Status = 'Rejected' THEN 1 ELSE 0 END)
```

```

/ NULLIF(COUNT(ef.Diary_number), 0), 2
) AS rejection_rate_pct
FROM Admin adm
JOIN "User" u ON u.User_ID = adm.User_ID
JOIN E_filing ef ON ef.Admin_ID = adm.Admin_ID
GROUP BY adm.Admin_ID, u.Name
HAVING ROUND(
100.0 * SUM(CASE WHEN ef.Status = 'Rejected' THEN 1 ELSE 0 END)
/ NULLIF(COUNT(ef.Diary_number), 0), 2) > 30
ORDER BY rejection_rate_pct DESC;

```

3. Courtroom related Queries:

→ These queries are based on daily life in courts. How daily all the system works in court

1) The court clerk prints today's schedule for Courtroom 3 — every case, its time slot, judge, and both lawyers.

```

SELECT
    cl.List_ID,
    cl.Date AS hearing_date,
    cl.Time AS hearing_time,
    cl.Courtroom_number,

    c.Case_ID,
    c.Case_title,
    c.Case_type,
    c.Current_status AS case_status,

    h.Hearing_ID,
    h.Purpose AS hearing_purpose,
    h.Verdict,
    h.Next_date,

    j_user.Name AS judge_name,

```

```

        j.Working_status                AS judge_status,

        p_user.Name                     AS prosecutor_name,
        pl.Firm                         AS prosecutor_firm,

        d_user.Name                     AS
defense_lawyer_name,
        dl.Firm                         AS defense_firm

FROM Cause_list cl

JOIN Hearing h          ON cl.List_ID      = h.List_ID

JOIN "Case" c          ON h.Case_ID       = c.Case_ID

JOIN Judge j           ON h.Judge_ID      = j.Judge_ID
JOIN "User" j_user     ON j.User_ID       = j_user.User_ID

JOIN Lawyer pl         ON h.Prosecutor_ID = pl.Bar_council_number
JOIN "User" p_user     ON pl.User_ID      = p_user.User_ID

JOIN Lawyer dl         ON h.Defense_lawyer_ID = dl.Bar_council_number
JOIN "User" d_user     ON dl.User_ID      = d_user.User_ID

WHERE
    cl.Courtroom_number = 3
    AND cl.Date          = CURRENT_DATE

ORDER BY
    cl.Time ASC;

```

2) Identifies cases that have been heard but have no next date scheduled and no verdict.

```

WITH Latest_Hearing AS (
    SELECT
        h.Case_ID,
        h.Hearing_ID,
        h.Judge_ID,

```

```

        h.Prosecutor_ID,
        h.Defense_lawyer_ID,
        h.Purpose,
        h.Verdict,
        h.Next_date,
        h.List_ID,
        ROW_NUMBER() OVER (
            PARTITION BY h.Case_ID
            ORDER BY h.List_ID DESC
        )
        ) AS rn
    FROM Hearing h
)

SELECT
    c.Case_ID,
    c.CNR,
    c.Case_title,
    c.Case_type,
    c.Current_status,
    c.Filing_date,

    lh.Hearing_ID AS last_hearing_id,
    lh.Purpose AS last_hearing_purpose,
    cl.Date AS last_hearing_date,
    cl.Time AS last_hearing_time,
    cl.Courtroom_number,

    j_user.Name AS judge_name,

    p_user.Name AS prosecutor_name,

    d_user.Name AS defense_lawyer_name,

    (CURRENT_DATE - cl.Date) AS days_since_last_hearing

FROM Latest_Hearing lh

JOIN "Case" c ON lh.Case_ID = c.Case_ID

LEFT JOIN Cause_list cl ON lh.List_ID = cl.List_ID

```

```

JOIN Judge j          ON lh.Judge_ID      = j.Judge_ID
JOIN "User" j_user    ON j.User_ID       = j_user.User_ID

JOIN Lawyer pl        ON lh.Prosecutor_ID = pl.Bar_council_number
JOIN "User" p_user    ON pl.User_ID      = p_user.User_ID

JOIN Lawyer dl        ON lh.Defense_lawyer_ID = dl.Bar_council_number
JOIN "User" d_user    ON dl.User_ID      = d_user.User_ID

WHERE
    lh.rn              = 1

    AND lh.Hearing_ID IS NOT NULL

    AND (lh.Next_date IS NULL
    OR lh.Next_date::text = '')

    AND (lh.Verdict IS NULL
    OR lh.Verdict::text = '')

    AND c.Current_status NOT IN ('Closed', 'Disposed', 'Resolved')

ORDER BY
    days_since_last_hearing DESC;

```

3) The court administrator checks which courtrooms are overloaded and which are underused to redistribute scheduling.

```

WITH courtroom_load AS (
    SELECT
        cl.Courtroom_number,
        COUNT(h.Hearing_ID)                AS total_hearings,
        COUNT(DISTINCT cl.Date)            AS
distinct_hearing_days,
        ROUND(
            COUNT(h.Hearing_ID)::NUMERIC
            / NULLIF(COUNT(DISTINCT cl.Date), 0), 2
        )                                  AS
avg_hearings_per_day,
        COUNT(DISTINCT h.Case_ID)          AS distinct_cases
    FROM Cause_list cl
    LEFT JOIN Hearing h ON h.List_ID = cl.List_ID
    GROUP BY cl.Courtroom_number

```

```

),

overall_avg AS (
    SELECT AVG(total_hearings)::NUMERIC(10,2)      AS avg_total
    FROM courtroom_load
)

SELECT
    cr.Courtroom_number,
    cr.total_hearings,
    cr.distinct_hearing_days,
    cr.avg_hearings_per_day,
    cr.distinct_cases,
    oa.avg_total      AS
court_average_hearings,

    CASE
    WHEN cr.total_hearings > oa.avg_total * 1.3      THEN 'OVERLOADED'
    WHEN cr.total_hearings < oa.avg_total * 0.7      THEN 'UNDERUSED'
    ELSE      'NORMAL'
    END      AS load_status

FROM courtroom_load cr
CROSS JOIN overall_avg oa
ORDER BY cr.total_hearings DESC;

```

4. E-Filings related Queries

→ These queries focus on the filings of the cases and IA applications.

1) Retrieves all e-filings submitted by a particular lawyer along with admin who processed them.

```

SELECT
    ef.Diary_number,
    ef.Document_title,
    ef.Document_type,
    ef.Submit_date,
    ef.Status,

    c.Case_ID,
    c.CNR,

```

```

        c.Case_title,
        c.Case_type,

        l_user.Name           AS lawyer_name,
        l.Firm                AS lawyer_firm,
        l.Working_status      AS lawyer_status,
        l.Bar_council_number,

        a_user.Name           AS admin_name,
        a.Post                 AS admin_post,
        a.Working_here_since  AS admin_since

FROM E_filing ef

JOIN "Case" c          ON ef.Petitioner_ID      = c.Case_ID

JOIN Lawyer l          ON ef.Petitioner_lawyer_ID = l.Bar_council_number
JOIN "User" l_user      ON l.User_ID            = l_user.User_ID

JOIN Admin a           ON ef.Admin_ID           = a.Admin_ID
JOIN "User" a_user      ON a.User_ID            = a_user.User_ID

WHERE

    l.Bar_council_number = :lawyer_bar_council_number

ORDER BY

    ef.Submit_date DESC;

```

2) Finds e-filings that remain pending beyond the expected processing window.

```

SELECT
    ef.Diary_number,
    ef.Document_title,
    ef.Document_type,
    ef.Submit_date,
    ef.Status,
    CURRENT_DATE - ef.Submit_date AS days_pending,
    ef.Petitioner_lawyer_ID       AS lawyer_bar_no,

```

```

        u_law.Name                                AS lawyer_name,
        ef.Admin_ID,
        u_adm.Name                                AS assigned_admin_name,
        adm.Post                                  AS admin_post,
        ef.Petitioner_ID,
        u_pet.Name                                AS petitioner_name

FROM E_filing ef

JOIN Lawyer    lw      ON lw.Bar_council_number = ef.Petitioner_lawyer_ID
JOIN "User"    u_law   ON u_law.User_ID        = lw.User_ID
JOIN Admin    adm     ON adm.Admin_ID          = ef.Admin_ID
JOIN "User"    u_adm   ON u_adm.User_ID        = adm.User_ID
JOIN Litigant lit     ON lit.Litigant_ID       = ef.Petitioner_ID
JOIN "User"    u_pet   ON u_pet.User_ID        = lit.User_ID

WHERE
    ef.Status = 'Under Review'
    AND ef.Submit_date < CURRENT_DATE - INTERVAL '30 days'

ORDER BY days_pending DESC;

```

3) Shows how many e-filings each admin has been assigned, useful for workload balancing

```

SELECT
    adm.Admin_ID,
    u.Name                                AS admin_name,
    adm.Post,
    COUNT(ef.Diary_number)                AS total_assigned,
    SUM(CASE WHEN ef.Status = 'Verified'
        THEN 1 ELSE 0 END)                AS verified,
    SUM(CASE WHEN ef.Status = 'Under Review'
        THEN 1 ELSE 0 END)                AS under_review,
    SUM(CASE WHEN ef.Status = 'Rejected'
        THEN 1 ELSE 0 END)                AS rejected,
    ROUND(
        100.0 * SUM(CASE WHEN ef.Status = 'Under Review'
            THEN 1 ELSE 0 END)
        / NULLIF(COUNT(ef.Diary_number), 0), 2
    )                                      AS pending_rate_pct

FROM Admin adm
JOIN "User" u      ON u.User_ID        = adm.User_ID

```

```

LEFT JOIN E_filing ef  ON ef.Admin_ID = adm.Admin_ID

GROUP BY
    adm.Admin_ID,
    u.Name,
    adm.Post

ORDER BY
    total_assigned DESC;

```

4) Calculates what percentage of e-filings each admin has rejected — useful for audit purposes

```

SELECT
    adm.Admin_ID,
    u.Name                                AS admin_name,
    adm.Post,
    adm.Working_here_since,
    COUNT(ef.Diary_number)                AS
total_filings_handled,
    SUM(CASE WHEN ef.Status = 'Rejected'
        THEN 1 ELSE 0 END)                AS rejected_count,
    ROUND(
        100.0 * SUM(CASE WHEN ef.Status = 'Rejected'
            THEN 1 ELSE 0 END)
        / NULLIF(COUNT(ef.Diary_number), 0), 2
    )                                     AS rejection_rate_pct

FROM Admin adm
JOIN "User" u      ON u.User_ID      = adm.User_ID
LEFT JOIN E_filing ef  ON ef.Admin_ID = adm.Admin_ID

GROUP BY
    adm.Admin_ID,
    u.Name,
    adm.Post,
    adm.Working_here_since

ORDER BY
    rejection_rate_pct DESC;

```